

Implementação da Política SCHED_BACKGROUND no Kernel Linux

Autor: Helian Silva
Disciplina: DCC403 – Projeto Final | UFRR

O Problema da Concorrência de Recursos

- Em ambientes de alta demanda, processos críticos (ex: Banco de Dados, Servidores Web) competem por tempo de CPU com tarefas utilitárias (ex: backups, compactação de logs, rotinas assíncronas).
- O escalonador padrão do Linux (CFS - Completely Fair Scheduler) tenta ser justo, garantindo fatias de tempo para todos.
- O Gargalo: Mesmo configurando processos utilitários com prioridade mínima (alta niceness), eles ainda causam preempção e roubam ciclos de processamento dos serviços essenciais, gerando sobrecarga e atrasos.

Solução Proposta

SCHED_BACKGROUND

- Uma nova classe de escalonamento nativa (identificador 7) desenvolvida para o Kernel do Linux 5.15.
- **Objetivo:** Criar um isolamento térmico e matemático absoluto para processos secundários.
- **Como funciona:** Atua no nível mais baixo da hierarquia do agendador (acima apenas da classe idle e abaixo da fair).
- **Regra de Ouro:** É uma política estritamente não-preemptiva. Tarefas em background só recebem ciclos de CPU se o sistema estiver absolutamente ocioso.

Lógica de Controle e o Arquivo **background.c**

- **kernel/sched/core.c**: Modificação nas funções **sched_init()** e **__sched_fork()** com **INIT_LIST_HEAD** para formatar listas encadeadas vazias e prevenir Page Faults por ponteiros nulos no boot.
- **background.c**: Implementação do núcleo da classe:
 - **enqueue_task** e **dequeue_task** para gerenciar a fila.
 - **pick_next_task** para repassar a CPU apenas se CFS e RT estiverem vazios.
 - **task_tick** implementando o mecanismo de Round-Robin para dividir recursos entre múltiplas tarefas background.

Metodologia de Validação

teste_sched.c:

Recebe o PID de um processo e utiliza a chamada de sistema `sched_setscheduler()` para forçar sua migração para a nova política `SCHED_BACKGROUND` (7). Em seguida, atua como um mecanismo de validação usando `sched_getscheduler()` para interrogar o Kernel e comprovar que a mudança foi aceita com sucesso e sem erros..

estressor.c:

Gera processamento intensivo alocando e multiplicando matrizes gigantes na memória. Utiliza as funções `gettimeofday` e `getrusage` para medir exatamente o tempo que a CPU passou calculando (User Time) e o tempo em que o processo ficou pausado aguardando a liberação da CPU (Wallclock), comprovando o isolamento na prática

Resultados

Cenário de Execução	Política Aplicada	Carga do Sistema (Concorrência)	Tempo de Parede (Wallclock)	Tempo de Usuário (User Time)
Linha de Base	CFS (Padrão)	Nenhuma (CPU Ociosa)	5.12 s	5.11 s
Prova de Isolamento	SCHED_BACKGROUND	Saturada (8 núcleos)	18.45 s	5.14 s

The background is a light gray color. It features several abstract line patterns. In the top-left corner, there are a few wavy, irregular lines. In the top-right corner, there is a series of concentric, hand-drawn circles. In the bottom-left corner, there are more wavy lines, some of which form a partial oval shape. In the bottom-right corner, there is a single, smooth, wavy line.

Fim.

Referências

LOVE, R. Linux Kernel Development. S.L.: Pearson Education India, 2018.

BOVET, D. P.; CESATI, M. Understanding the Linux kernel. Beijing Etc.: O'reilly, 2006.

ALESSANDRO RUBINI. Linux device drivers. Sebastopol, Ca: O'reilly & Associates, Inc, 1998.